Source files and execution

CSC103 Programming Fundamentals / Lecture 02

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Create Welcome.java. Print your name and the result of 9 * 4 on separate lines. Compile and execute it.

Create a source file whose public class matches its filename.

Learning objectives

- Create and compile a Java source file
- Explain the Java execution pipeline
- Classify syntax, runtime and logic errors

CLO-1
A first Java source file
Compilation and execution
Three error categories
Files and working directories

A first Java source file

- A public class name matches its .java filename.
- The conventional entry point is public static void main(String[] args).

Example

```
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello, Java");
  }
}
```

Compilation and execution

- javac translates source code into JVM bytecode.
- java starts the JVM, which loads and links classes, then executes the program.

Example

```
javac Hello.java
java Hello

Hello.java   source
Hello.class  bytecode
```

Three error categories

- Syntax and type errors prevent successful compilation.
- Runtime exceptions interrupt execution. Logic errors produce an incorrect result even when execution finishes.

Example

```
Syntax: missing ;
Runtime: Integer.parseInt("cat")
Logic: area = length + width
```

Files and working directories

- A terminal command runs relative to its working directory.
- An editor or IDE helps save, build and run code, but the Java compiler still enforces language rules.

Example

```
1. Create a course folder.
2. Save Hello.java as plain text.
3. Open a terminal in that folder.
4. Compile, then run.
```

What compilation checks

- The compiler checks syntax and type consistency before producing bytecode.
- It cannot generally decide whether your formula answers the problem correctly.
- A successful build therefore needs a separate execution and result check.

Example

```
int n = "five";  // incompatible types
int area = 5 + 3; // legal, but wrong for a 5 by 3 area
```

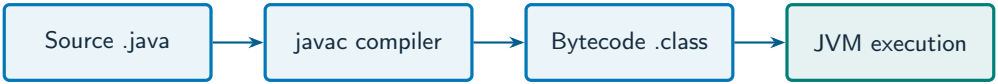
Reading an error message

- A diagnostic normally identifies a file, line and kind of problem.
- Start with the earliest relevant error because one mistake can trigger several messages.
- Recompile after a small correction to see which messages remain.

Example

```
Missing semicolon near a println statement:
Check that statement and the previous line.
```

The Java execution pipeline



What to notice

Compilation and execution are different stages; each needs its own check.

Key distinctions

Stage	Input	Result
Edit	Plain-text source	Welcome.java
Compile	javac Welcome.java	Welcome.class or diagnostics
Load and link	java Welcome	JVM prepares classes
Execute	main statements	Console output

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
System.out.println("Welcome to CSC103");
System.out.println(7 + 5);
```

First example: result

Welcome to CSC103
12
The first statement prints text. The second prints an arithmetic result.

Three error categories
Files and working directories

Guided practice

Create Welcome.java. Print your name and the result of 9 * 4 on separate lines. Compile and execute it.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Create a source file whose public class matches its filename.
- Put the two output statements inside main.
- Compile the file, then run the class without a .class extension.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```java
public class Solution02 {
  public static void main(String[] args) throws Exception {
    System.out.println("Amina");
    System.out.println(9 * 4);
  }
}
```

Execution trace

Statement	Evaluation	Console line
println("Amina")	String literal	Amina
println(9 * 4)	9 multiplied by 4	36
End of main	Normal return	No additional line

Follow the changing state in execution order.

Solution result

Amina
36

Why this result follows
Compile the file, then run the class without a .class extension.

A common incorrect approach

File name: Start.java
public class Begin { }

Example

```
Rename the file Begin.java or the public class Start. Java requires the public top-level class
     and filename to match.
```

Boundary and failure checks

Check the stated input assumptions.
Create one example of each error category. Record the error message or wrong result and its correction.

The public class must be Welcome. Place both println statements inside main. Expected second line: 36.

Check your understanding

Does java Hello compile Hello.java in the demonstrated two-step workflow? What file does javac create?

Write a prediction and explain the rule that supports it.

Answer and explanation

No. java Hello runs the class. javac Hello.java creates Hello.class. Java also has a separate source-file launch mode, outside this demonstration.

Rename the file Begin.java or the public class Start. Java requires the public top-level class and filename to match.

Compare cases and predict outcomes

Observation	Stage	Next check
Missing semicolon	Compilation	Inspect this and the preceding line
Class not found	Launch / loading	Class name and classpath
Wrong printed total	Execution / logic	Formula and test input

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

A public class named Receipt is saved as Bill.java. After renaming it, the program prints 5 + 3 when the required area is 5 by 3. Identify both corrections and the commands.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```java
// Save as Receipt.java
public class Receipt {
  public static void main(String[] args) {
    System.out.println(5 * 3);
  }
}
// javac Receipt.java
// java Receipt
```

- The public class and source filename must agree.
- Changing + to * fixes the computation; successful compilation alone cannot establish that.
- Expected output: 15.

Java program phases and runtime roles

- javac checks source and emits class-file bytecode; the java launcher starts execution through a JVM.
- Loading, verification and execution are distinct runtime responsibilities. JIT compilation works on bytecode, not Java source.
- Use a JDK for development. Modern JDK layouts no longer use the old rt.jar structure shown in the source.

Source: supplied ISB campus slides. See speaker notes and ISB_Source_Map.md.

Worked problem: Java program phases and runtime roles

Task

Build and run a small program, then identify which phase detects a missing semicolon and which phase prints the greeting.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture02 {
  public static void main(String[] args) throws Exception {
    System.out.println("Hello, World");
  }

}
```

Worked trace and expected result

Stage	Artifact or action	Evidence
Compile	javac IsbLecture02.java	Class file or diagnostic
Load and verify	Prepare class bytecode	Valid class structure required
Execute	Run main	Hello, World

Expected console output

```
Hello, World
```

Extend the ISB example

Practice

Does bytecode verification prove that the formula in a program is correct?

Explain your reasoning

No. Verification checks structural and type-related constraints; a valid program can still compute the wrong result.

Lecture summary

- and compile a Java source file
- the Java execution pipeline
- syntax, runtime and logic errors

- Create a source file whose public class matches its filename.
- Put the two output statements inside main.
- Compile the file, then run the class without a .class extension.

After-class practice

Create one example of each error category. Record the error message or wrong result and its correction.

Syllabus reading
Deitel Ch 1
Next lecture: Java syntax and program style